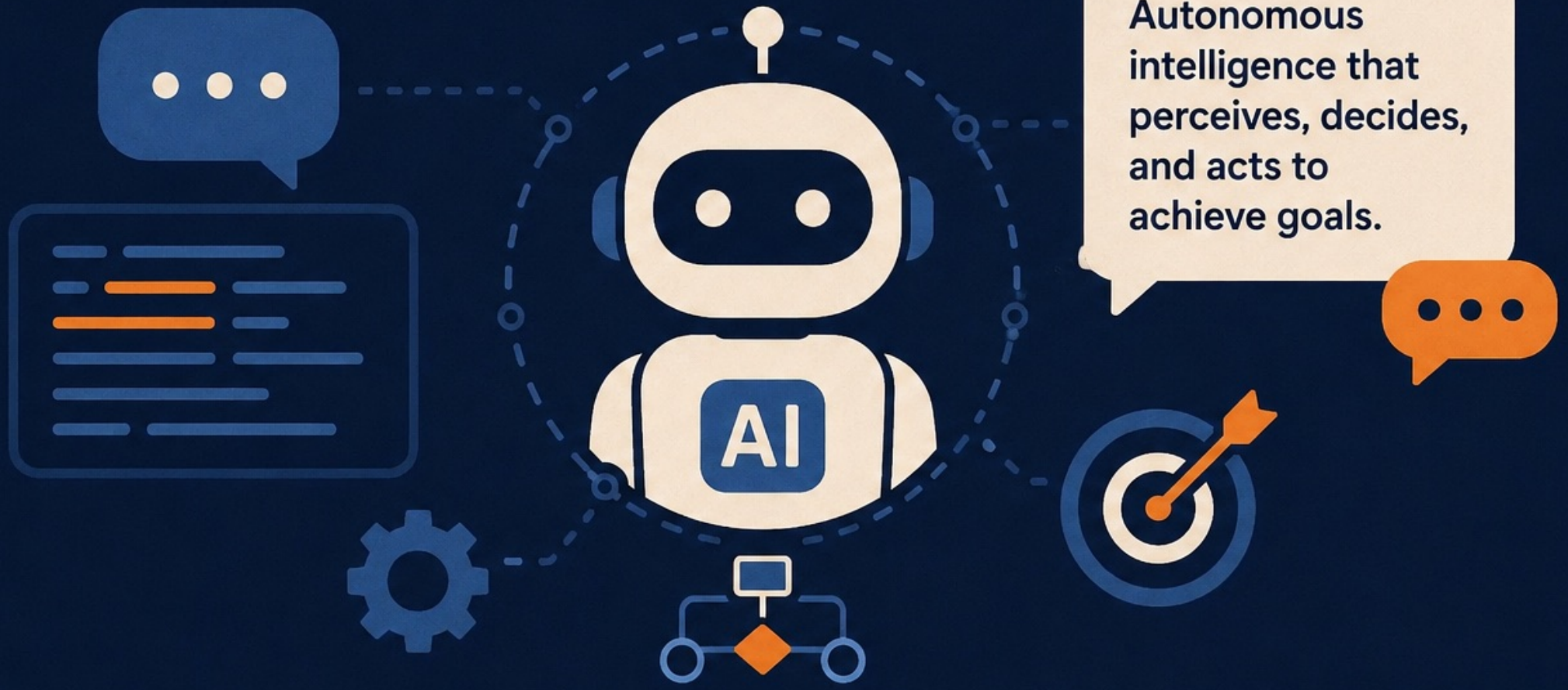
AI AGENTS



PERCEIVE | REASON | DECIDE | ACT | ACHIEVE

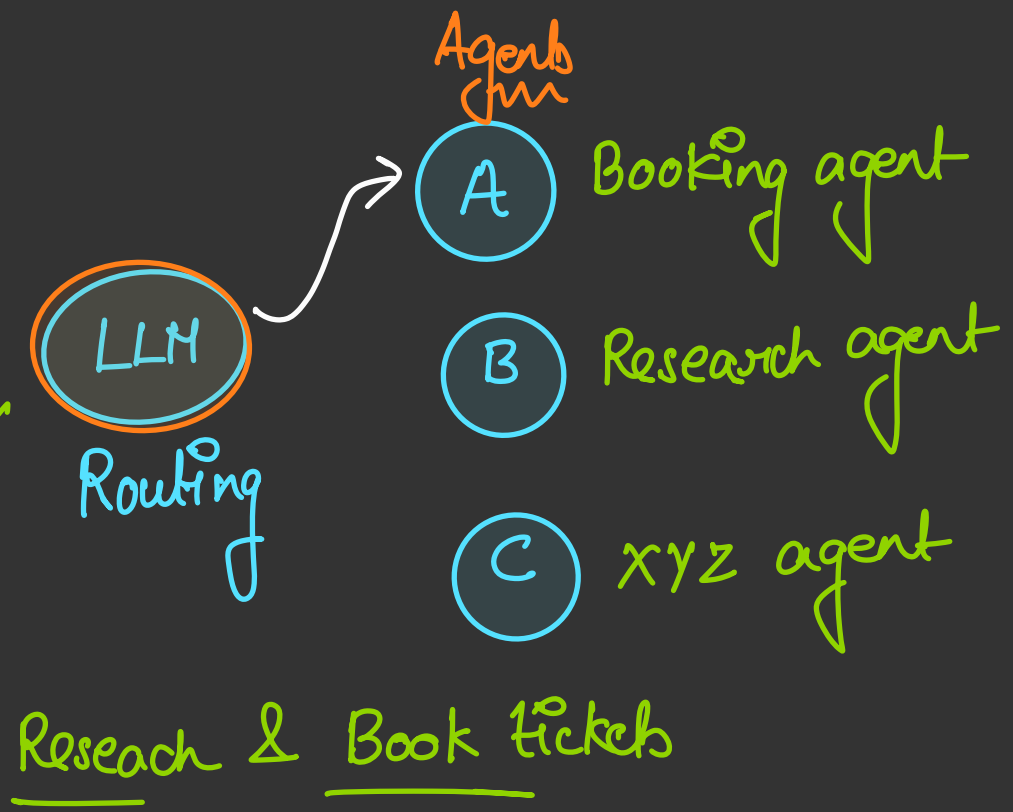
Lecture: Agents Workflow and MCP Theory

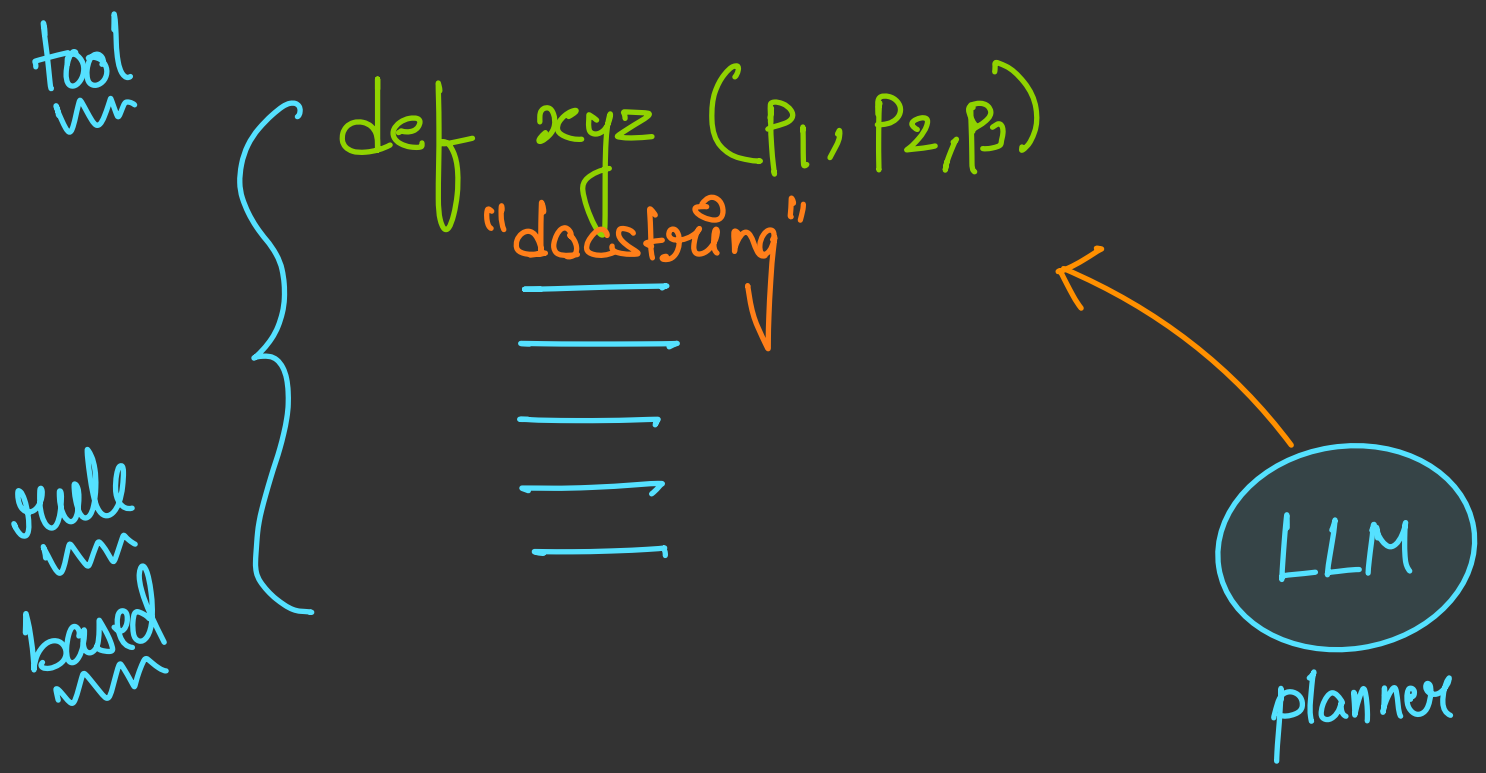
#Agenda

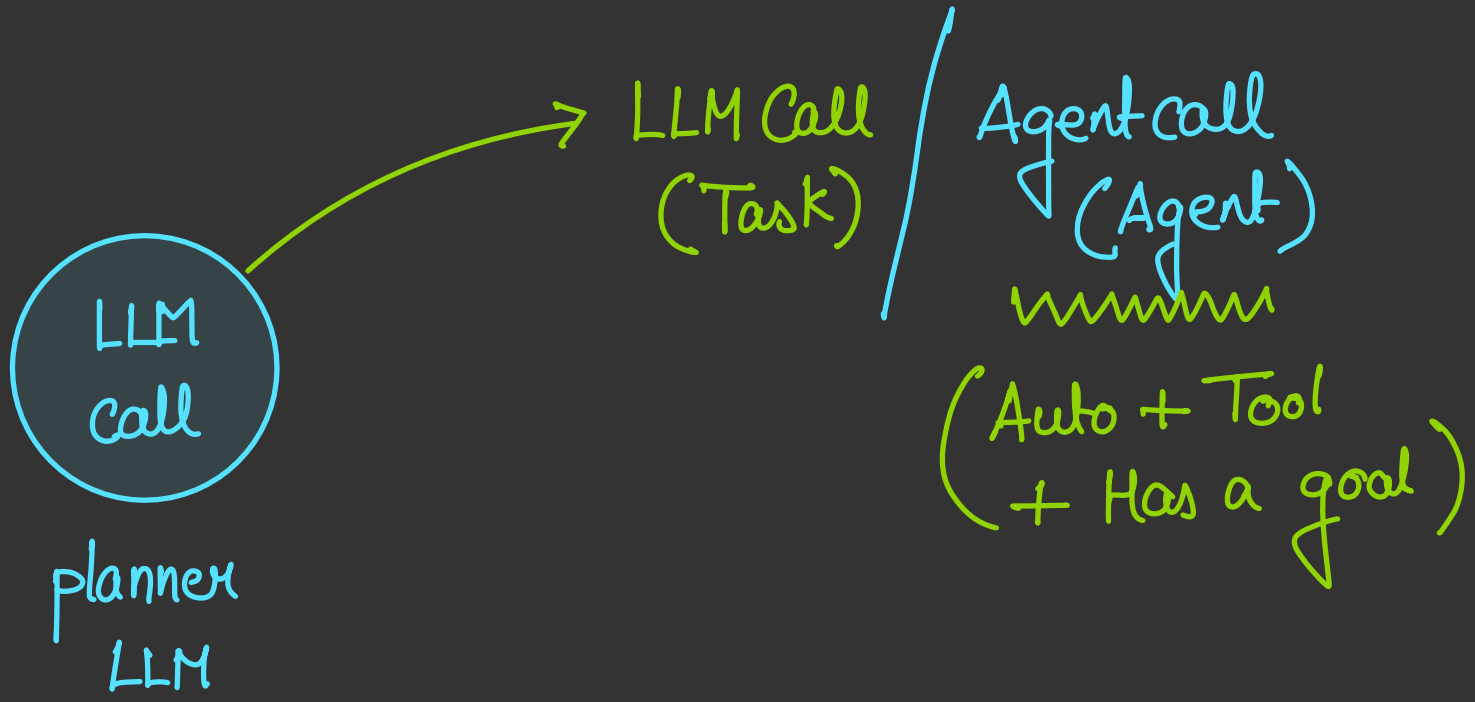
1) The Agent Workflow w/ patterns
2) MCP Theory

Agents Workflow

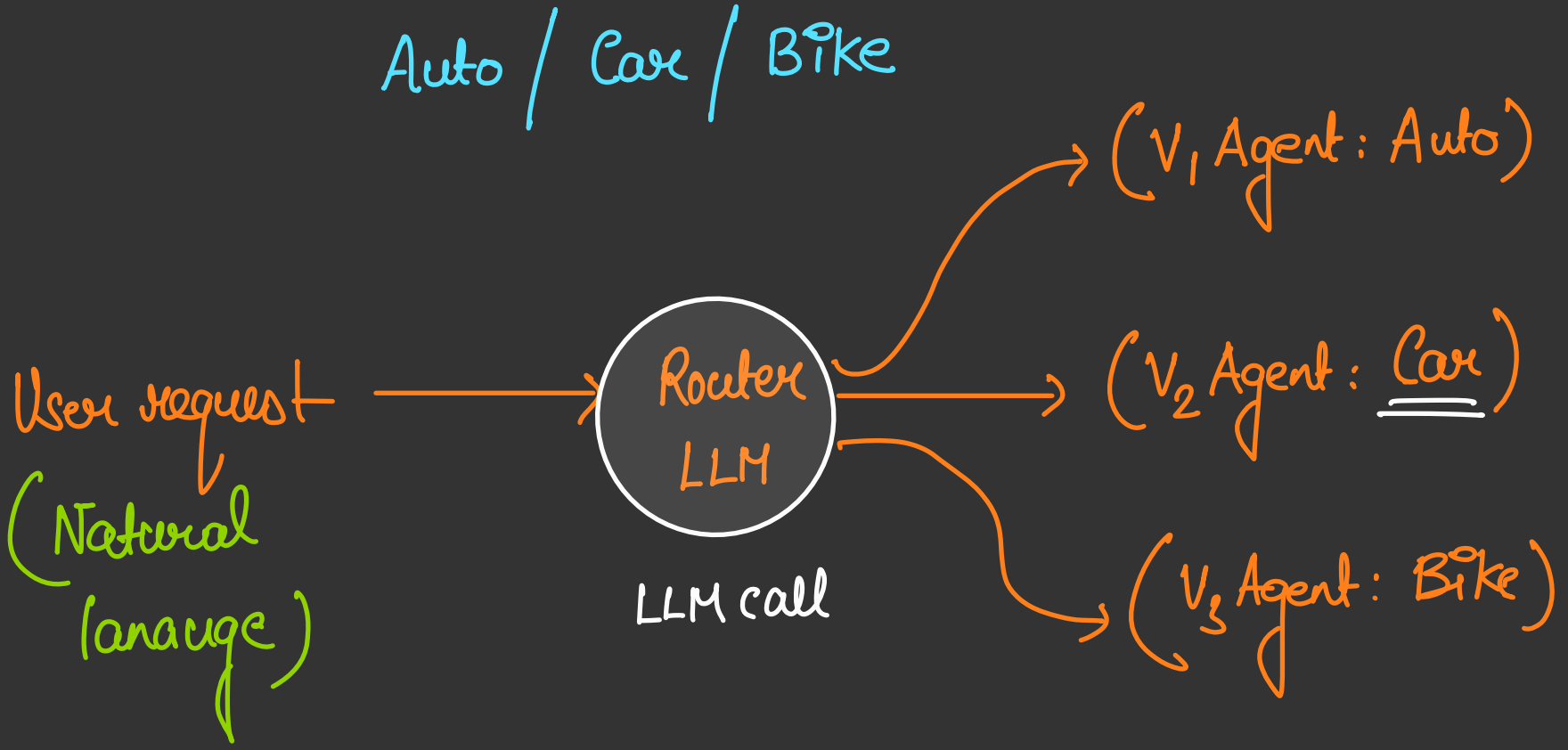
1) prompt Chaining (Sequential)
2) Routing (gateway redirection)
3) parallelization
4) Evaluator optimizer
5) Orchestrator – Worker




tool
def xyz (p1, p2, p3)
"docstring"
rule based
LLM
planner


LLM Call (Task)
Agent call (Agent)
(Auto + Tool + Has a goal)
LLM Call
planner LLM

Ola/Uber Support System



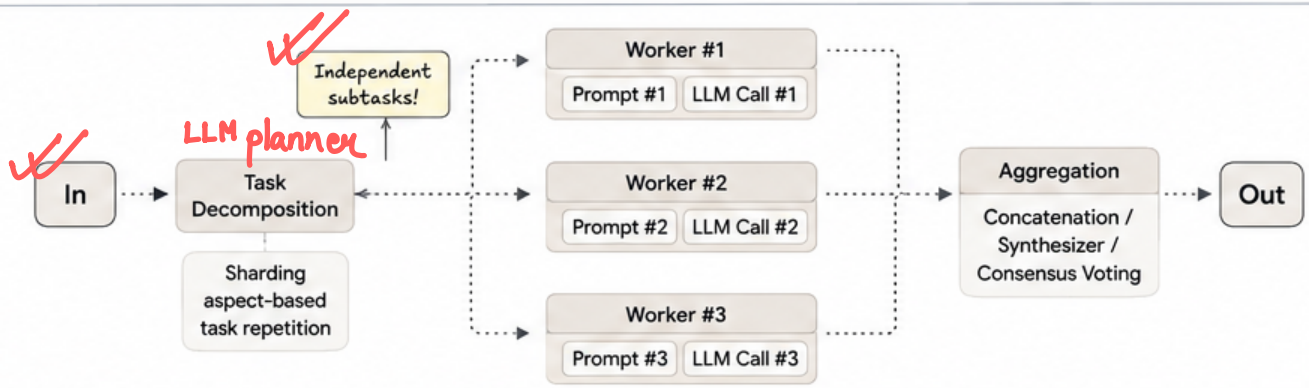
WORKFLOW 3

Parallelization

Scatter–Gather for Concurrent LLM Calls

1. DEFINITION

- Parallelization distributes LLM work across multiple concurrent calls whose outputs are aggregated into a single result.
- It follows the scatter–gather pattern from distributed computing: split the task, process in parallel, consolidate results.
- **Prerequisite:** subtasks must be **largely independent of one another.** If the output of **subtask A is required as input to subtask B,** that dependency is sequential and belongs in a prompt chain (Workflow 1).



2. HOW IT DIFFERS FROM OTHER PATTERNS

Prompt Chaining (Workflow 1)	Routing (Workflow 2)	Parallelization (Workflow 3)
Sequential	**Branching**	**Concurrent**
Each step depends on the previous step's output.	One path is selected, others are skipped.	Multiple paths execute simultaneously, all outputs are used.

3. TWO MAJOR PATTERNS

A. SECTIONING

Break a task into independent subtasks with different responsibilities. Each subtask handles a distinct concern.

Example:
One call generates a response while another screens it for policy violations.

B. VOTING

Run the same task multiple times (possibly with varied prompts or temperatures) to get diverse outputs.

Example:
Three independent reviewers each flag code vulnerabilities; an issue is confirmed only if two or more agree.

4. AGGREGATION STRATEGIES

- **Concatenation** – Join outputs into a single result.
- **Comparison & Selection** – Pick the best output.
- **Voting / Majority Rule** – Use consensus or thresholds.
- **Synthesizer LLM** – A dedicated model merges results.

5. TRADE-OFFS

- **Cost:** Scales linearly with fan-out width (more parallel calls = higher cost).
- **Latency:** Wall-clock time can decrease because calls run concurrently.
- **Quality:** Gains come from separation of concerns (sectioning) or redundancy (voting), not from speed alone.

6. WHEN TO USE

Use parallelization when the divided subtasks can be parallelized for speed, or when multiple perspectives or attempts are needed for higher confidence results.

For complex tasks with multiple considerations, LLMs generally perform better when each consideration is handled by a separate LLM call, allowing focused attention on each specific aspect.

7. EXAMPLES

- **Sectioning**
  - Implementing guardrails: one model generates responses while another screens for inappropriate content or requests.
  - Automating evals: each call evaluates a different aspect of model performance on a given prompt.
- **Voting**
  - Review code for vulnerabilities using multiple prompts.
  - Evaluate content appropriateness with multiple prompts or different vote thresholds.

Key Takeaway: Parallelization uses concurrency to increase throughput and/or confidence through independent execution and smart aggregation.

WORKFLOW 3

4. Text2SQL Parallelization Implementation

Generate SQL → Verify in Parallel (Vote + Safety) → Repair if Needed → Execute

1 GENERATE SQL

Use existing prompt chain (text2sql_chain) from Workflows 1 & 2

Output: SQL Query

2 PARALLEL VERIFICATION

Run 4 independent checks concurrently

Output: Votes + Safety Result

3 DECISION

Aggregate results

Pass all checks?

NO

Yes

EXECUTE SQL

Run query on database (execute_sql)

Output: Query Results

RUN IN PARALLEL (ThreadPoolExecutor)

Critic 1: Schema Compliance

Verifies all tables and columns exist in the schema

Returns JSON: {passes, issues[]}

Critic 2: Aggregation Logic

Checks GROUP BY, HAVING, and use of aggregates match the question

Returns JSON: {passes, issues[]}

Critic 3: Join Correctness

Validates JOIN conditions, missing joins, and cartesian product risks

Returns JSON: {passes, issues[]}

Safety Screener (Sectioning)

Screens the question for destructive, unbounded, or data-exfiltration risks

Returns JSON: {safe, risk_level, flags[], recommendation}

AGGREGATE RESULTS

- Critics: Majority vote (threshold = 2 of 3)
- Safety: recommendation must not be "block"
- Combine issues from failing critics

Example Vote (threshold = 2)

✓ ✓ ✗ = PASS

✗ ✗ ✓ = FAIL

5 REPAIR & RE-VERIFY

Fix SQL using aggregated issues and re-run parallel verification

Re-verify

Max Repairs Reached?

NO

YES

Still Failing

Execute with Warning

PATTERNS USED

Voting Pattern

Three independent critics evaluate the SQL on different dimensions. Majority vote determines pass/fail.

Sectioning Pattern

Safety screener runs in parallel with SQL generation and critics, handling a distinct responsibility.

PASS CRITERIA

- ✓ Safety screener recommendation is not "block"
- ✓ At least 2 of 3 critics must pass

OUTPUTS

On Success
Final SQL + Query Results + Full Trace

On Failure
Aggregated issues + Trace + Warning (if executed)

KEY BENEFITS

Higher Confidence
Multiple independent validations catch more issues

Lower Latency
All checks run concurrently (parallel execution)

Auto-Repair
Aggregated issues guide targeted SQL fixes

Full Auditability
Complete trace of generation, verification, repairs, and execution

WORKFLOW 4

Evaluator-Optimizer

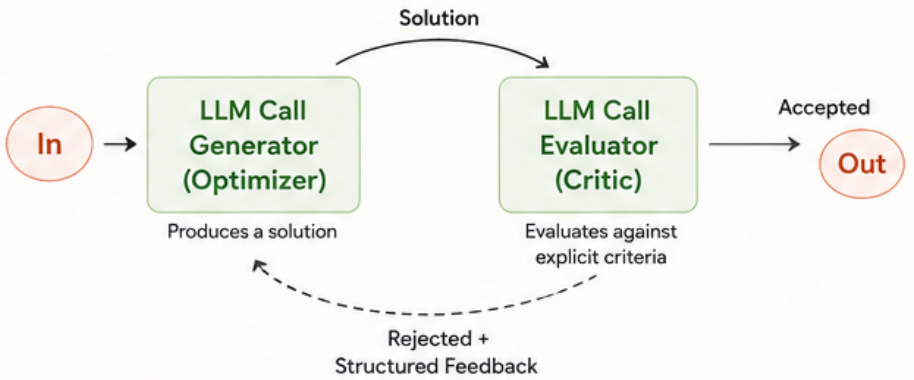
Iterative Feedback Loop for Measurable Improvement

1. DEFINITION

- **Evaluator-Optimizer** is a two-role iterative loop: one LLM generates output (the Optimizer), another LLM evaluates it against explicit criteria (the Evaluator). If evaluation fails, structured feedback flows back into the generator for a bounded number of retries.

Core Idea:
Generate → Evaluate → Feedback → Refine → Repeat (until accepted or limit reached)

Iterative Improvement Loop



WHY SEPARATE THE ROLES?

- A single LLM that generates AND self-critiques suffers from anchoring bias—it's reluctant to contradict its own output.
- Splitting generation from evaluation creates genuine adversarial tension: the evaluator has no investment in the output and evaluates more objectively.

2. HOW IT DIFFERS FROM OTHER WORKFLOWS

Workflow 1 Prompt Chaining

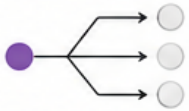
Linear pipeline. Each step runs once, no backflow.

If Stage 3 produces bad SQL, Stage 4 flags it, but there is no mechanism to return to Stage 3 with feedback.

Workflow 2 Routing

Single classification decision, no iteration.

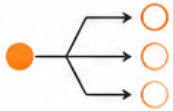
Sends work somewhere and is done.



Workflow 3 Parallelization

Multiple evaluators run simultaneously, but the evaluated artifact is fixed.

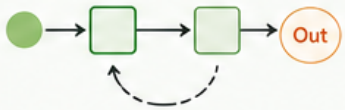
No regeneration loop.



Workflow 4 Evaluator-Optimizer

Closed feedback loop. The evaluator's output becomes the generator's next input.

Supports genuine iterative improvement.



3. WHEN TO USE IT

1. There are clear, articulable evaluation criteria.
2. LLM output demonstrably improves when given specific feedback.
3. A single generation pass is not reliably sufficient.

Common Signals:

- Output quality is highly variable across runs.
- Task involves satisfying multiple constraints simultaneously.

4. KEY DESIGN CONSTRAINTS

Bounded Loop
Limit iterations (max retries) to control cost and latency.

Structured Evaluation
Use structured criteria and outputs (JSON, checklists, scores, PASS/FAIL), not freeform prose.

Deterministic Stop Condition
Explicit PASS/FAIL (or threshold), not a vague quality score.

Actionable Feedback
Feedback must be specific and structured to drive improvement.

5. KEY TAKEAWAY

Evaluator-Optimizer is the first workflow that creates a true improvement loop—combining objective evaluation with targeted feedback to iteratively converge on higher-quality solutions.

WORKFLOW 4

Text2SQL Evaluator-Optimizer in Action

Iterative loop: Generate → Evaluate → Feedback → Revise (repeat) → Accept

1. WHY EVALUATOR-OPTIMIZER FITS TEXT2SQL

SQL can fail in many independent ways a single pass often misses.

Common Failure Modes

- **Schema hallucination** (bad columns/tables)
- **Incorrect or missing joins** (cartesian products or dropped rows)
- **Missing GROUP BY** clauses
- **Wrong aggregation functions** (COUNT vs. SUM, etc.)
- **Filter logic errors** (WHERE vs. HAVING, wrong operators)

2. HOW IT IMPROVES OVER PROMPT CHAINING (Workfiow 1)

Prompt Chaining (Linear)	Evaluator-Optimizer (Iterative)
Generate SQL	Generate SQL
↓	↓
Verify (single stage)	Multi-dimensional Evaluation
↓	↓
Repair (gets issue list only)	Full feedback (reasoning + fixes)
↓	↓
End (no re-verification)	Revise SQL
	↓
	Re-evaluate & Re-execute
✗ Issues introduced in repair may go undetected.	✓ True feedback loop catches new issues and converges.

EVALUATOR-OPTIMIZER LOOP

1 GENERATE SQL (Optimizer)

LLM generates SQL from question + schema (or revises using feedback).

Output: SQL Query

→

2 EVALUATE SQL (Evaluator)

LLM evaluates on 5 independent dimensions.

Output: Evaluation (Structured JSON)

→

3 VALIDATE EXECUTION (Deterministic)

Execute SQL against real database.

Output: Execution Result / Error

→

4 DECISION

Do both evaluation AND execution pass?

- **YES** — Accept & Return Results
- **NO** — Build Feedback & Retry

Repeat until accepted or max retries reached

5 FEEDBACK TO GENERATOR

Combine evaluator's repair instructions for ALL failed dimensions + ground-truth **execution errors** (if any).
Feed back to the generator for the next attempt.

EVALUATION DIMENSIONS (5)

1. Syntax Validity	2. Schema Compliance	3. Join Correctness	4. Aggregation Correctness	5. Semantic Alignment
Is the SQL syntactically correct?	Do all referenced tables and columns exist?	Are JOIN conditions correct and complete? Any cartesian risks?	Are GROUP BY clauses valid? Are aggregation functions appropriate?	Does the SQL answer the original question as intended?

★ **Goal:** Overall evaluation must PASS on all (or as configured) dimensions + successful execution.

3. IMPLEMENTATION COMPONENTS

SQL Generator (Optimizer)

- First call: generate from schema + question
- Later calls: **revise** using structured feedback
- Returns: **SQL query only**

SQL Evaluator (Critic)

- LLM checks 5 dimensions independently
- Returns structured JSON with:
  - Per-dimension pass/fail
  - Issue description
  - Repair instructions
  - Overall PASS/FAIL + confidence

Execution Validator (Deterministic)

- Execute against **real database**
- Returns **success/failure + results or error**
- Provides ground-truth signal

4. LOOP BEHAVIOR

- **Bounded retries** (max_retries)
- Deterministic stopping condition:
  - Evaluation overall = **PASS**
  - Execution = **Success**
- Structured feedback drives improvement
- Exhaust retries → **return** best effort + history

OUTPUTS

On Success	On Failure (Max Retries)
• Final SQL	• Last SQL attempt
• Query Results	• Last evaluation + errors
• Attempt count	• Execution error (if any)
• Full history trace	• Full attempt history

KEY TAKEAWAYS

Closed Feedback Loop
First pattern with true iteration—evaluator output becomes next input for the generator.

Higher Reliability
Re-evaluation after each repair catches newly introduced problems.

Multi-Signal Validation
Combines LLM judgment across multiple dimensions with real execution for maximum rigor.

Measurable Improvement
Converges toward correct, robust SQL within a bounded number of retries.

Production-Ready
Deterministic stopping, structured outputs, and full trace for observability.

WORKFLOW 5 Orchestrator-Worker

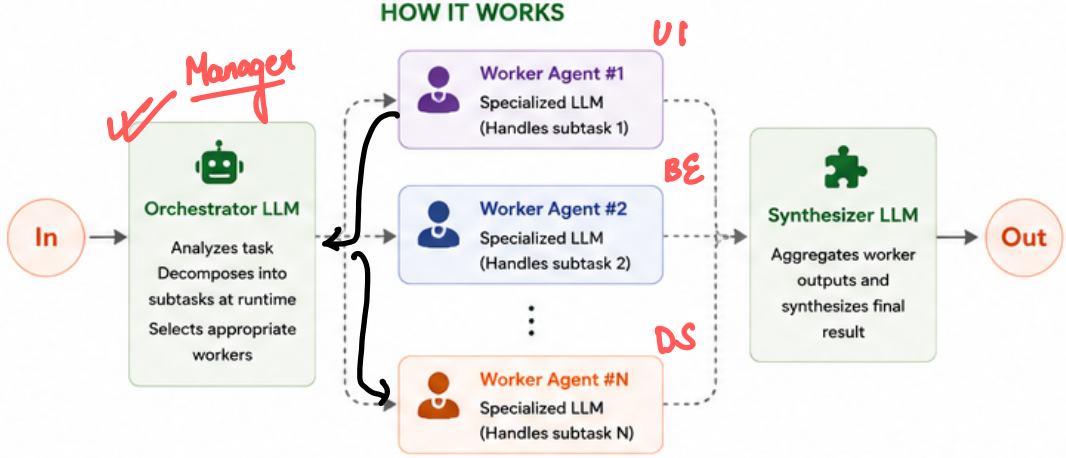
Dynamic Decomposition • Delegation • Synthesis

1. DEFINITION

A two-tier delegation pattern where a central Orchestrator LLM analyzes a complex task, dynamically decomposes it into subtasks at runtime, delegates each subtask to specialized Worker LLMs, and synthesizes their outputs into a final result.

Key Idea: Task decomposition is LLM-driven at runtime, not hardcoded.

HOW IT WORKS



Number of workers and subtasks are determined by the orchestrator based on the specific input.

5. WHEN TO USE IT

- Subtasks cannot be predicted in advance.
- Different inputs require different decompositions.
- Task requires multiple distinct specialized perspectives that must be coordinated.

Common Signals

- Task has variable structure.
- Number of steps depends on input complexity.
- Single prompt consistently misses some aspect of complex inputs.

2. HOW IT DIFFERS FROM OTHER WORKFLOWS

Workflow 1 Prompt Chaining
Fixed pipeline – stages are predetermined at design time.

Workflow 2 Routing
Single classification, single dispatch. Sends work to one path only.

Workflow 3 Parallelization
Predefined subtasks run in parallel. Workers are fixed regardless of input.

Workflow 4 Evaluator-Optimizer
Iterative refinement of a single artifact. Not a decomposition pattern.

Orchestrator-Worker: Dynamically decides **WHAT** subtasks are needed and **WHICH** workers to use for each specific input.

3. ORCHESTRATOR- WORKER PROCESS (HIGH LEVEL)

1. **Analyze** — Orchestrator analyzes the input and understands the overall goal.
2. **Decompose** — Break down the task into the necessary subtasks at runtime.
3. **Delegate** — Select appropriate worker agents and assign each subtask.
4. **Execute** — Workers complete their assigned subtasks (can run in parallel).
5. **Synthesize** — Synthesizer aggregates worker outputs into a coherent final result.

Example (Research Task)

- Orchestrator plans research
- Workers research web, analyze data, summarize papers, extract facts
- Synthesizer writes comprehensive report

6. TRADE-OFFS

Higher Latency & Cost
Orchestrator planning + multiple worker calls + synthesis adds overhead.

Coordination Complexity
Managing dynamic workers and consolidating outputs increases system complexity.

Quality Depends on Orchestrator
Final result quality relies heavily on how well the orchestrator decomposes the task.

Not Always Necessary
If subtasks are predictable and fixed, parallelization is simpler, cheaper, and faster.

KEY TAKEAWAYS

Dynamic & Adaptive
Decomposes tasks based on the specific input at runtime.

Leverages Specialization
Uses the right worker for each subtask to maximize quality.

Comprehensive Results
Combines multiple perspectives into a complete, coherent solution.

Best for Complex, Unpredictable Tasks
Ideal when the path forward isn't known in advance.

WORKFLOW 5

3. Orchestrator-Worker in Text2SQL

Dynamic Decomposition • Specialized Workers • Intelligent Synthesis

1. WHY THIS PATTERN FOR SQL?

Complex analytical questions have multiple interlocking requirements that a single generation pass frequently misses (joins, aggregation, filters, subqueries, etc.).

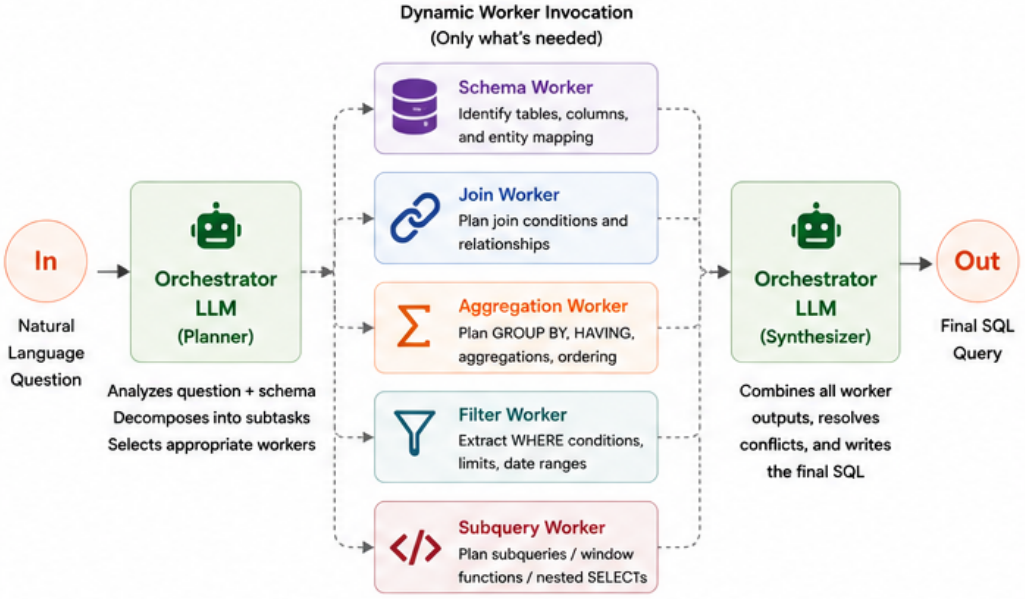
Common Failure Modes in Single-Pass Generation

- **Schema hallucination** (non-existent tables/columns)
- **Incorrect or missing joins** (cartesian products)
- **Missing GROUP BY / wrong aggregation**
- **Filter logic errors** (WHERE vs. HAVING, operators)
- **Subquery / window function errors**

2. HOW IT DIFFERS FROM OTHER WORKFLOWS

Workflow 1 Prompt Chaining	Fixed pipeline. Stages are predetermined at design time. S1 → S2 → S3 → ...
Workflow 2 Routing	Single classification, single dispatch. Sends work to one path only.
Workflow 3 Parallelization	Predefined subtasks run in parallel. Workers are fixed regardless of input.
Workflow 4 Evaluator-Optimizer	Generate → Evaluate
Workflow 5 Orchestrator-Worker	Dynamic decomposition at runtime. Orchestrator decides WHAT subtasks are needed and WHICH workers to use.

ORCHESTRATOR–WORKER ARCHITECTURE



The orchestrator decides tasks at runtime based on the specific input and schema.

4. IMPLEMENTATION OVERVIEW

1 Orchestrator Plans — `orchestrate_sql_plan()`

- Analyze question + schema Assess complexity
- Decide which workers are needed
- Produce structured task plan (JSON)

2 Workers Execute — Specialized worker LLMs

- Each worker gets a focused assignment + schema
- Work in parallel (where independent)
- Return structured JSON outputs

3 Orchestrator Synthesizes — `synthesize_sql()`

- Combine all worker outputs
- Resolve conflicts & gaps
- Write a single, coherent MySQL SELECT query

4 Execute & Return — `execute_sql()`

- Run SQL against database
- Return results or error
- Includes full trace: plan, worker outputs, SQL, execution status

EXAMPLE QUESTION (COMPLEX)

> "Show the top 3 departments by headcount where average salary exceeds 60000, including the department name, employee count, average salary, and the name of the highest-paid employee in each department."

Requires:

- Table identification & joins
- Aggregation + GROUP BY + HAVING
- Subquery/window for highest-paid employee
- Filters and ordering + LIMIT

KEY BENEFITS

Handles complexity
Breaks complex questions into focused, manageable pieces.

Higher accuracy
Each worker reasons about one concern with full attention.

Adaptive
Invokes only the workers needed for each specific query.

Maintainable & extensible
Add new specialized workers as needs evolve.

5. TRADE-OFFS

Extra Overhead planning + nufders	Orchestrator planning + multiple workers increase latency and cost.
Coordination Complexity	More moving parts and potential for inconsistent worker outputs.
Quality Depends on Orchestrator	If the decomposition is poor, final result suffers.
Not Aᵇʸᵃᵍᵃˢ Necesary	If subtasks are predictable and fixed, parallelization (Workflow 3) may be simpler and cheaper.

WHEN TO USE

- Subtasks cannot be predicted in advance
- Different inputs require different decompositions
- Task has variable structure
- Number of steps depends on input complexity
- Single prompt consistently misses some aspect of complex inputs

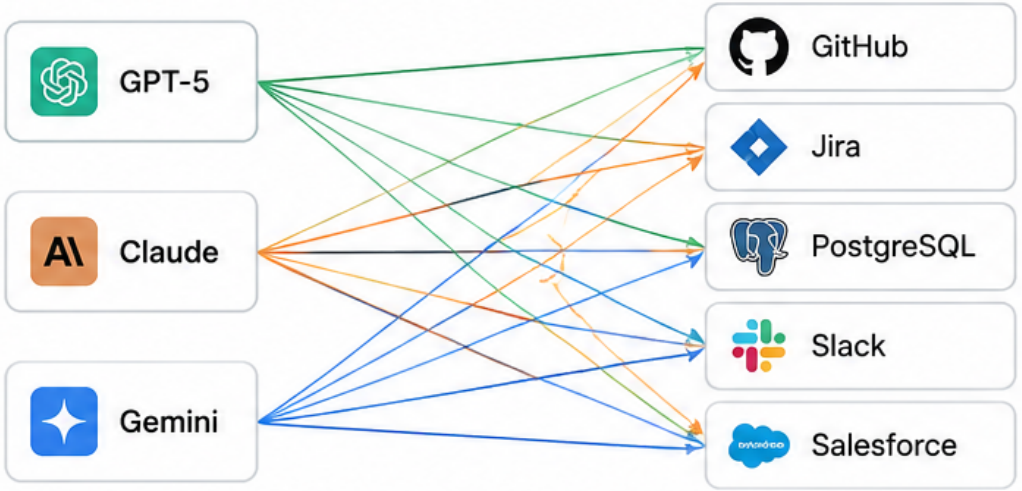
01 Lesson 1: Why MCP Exists (Model Context Protocol)

Before learning MCP, understand the problem.

Models
- GPT-5
- Claude
- Gemini

Tools
- GitHub
- Jira
- PostgreSQL
- Slack
- Salesforce

Without MCP, every model needs its own integration with every tool.



M N

If:
M = models
N = tools

then:

Integrations = M × N

Example:

3 models × 5 tools = 15 integrations

Every integration requires:

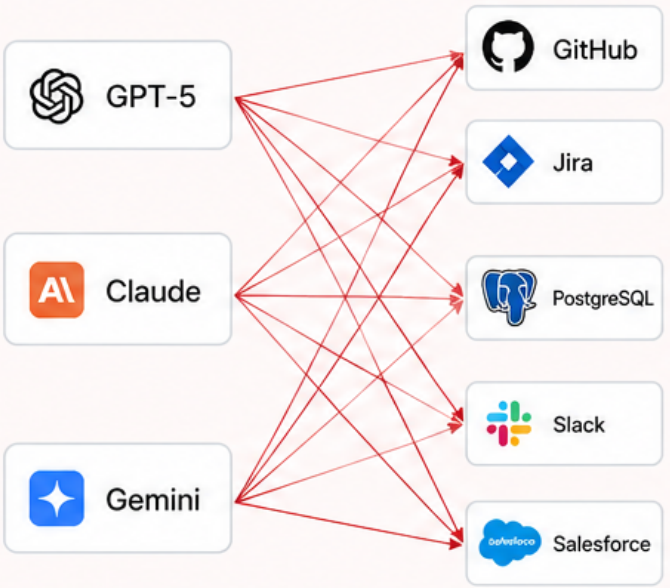
- Schema definition
- API handling
- Authentication
- Error handling
- Maintenance

This is the M × N Problem.

02 Lesson 2: MCP's Big Idea

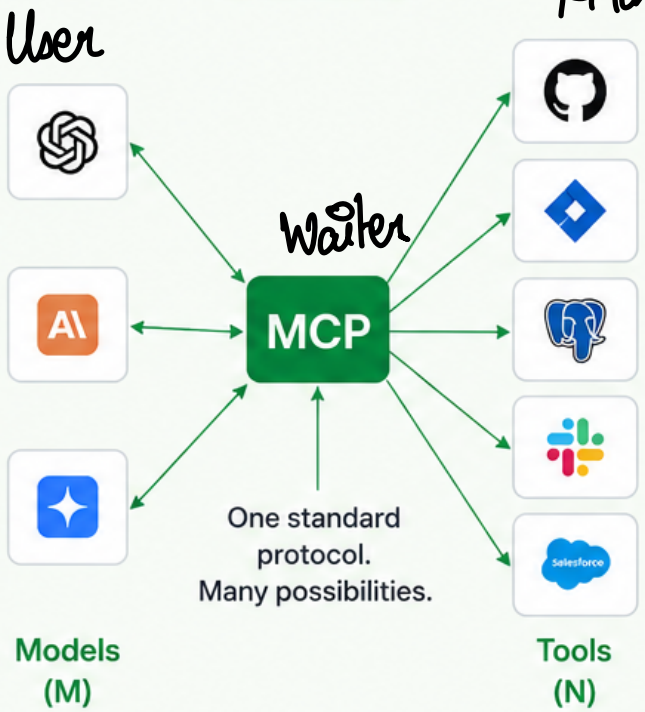
MCP introduces a standard protocol between models and tools.

Instead of (Without MCP)



Many point-to-point integrations (M × N)

With MCP



Single protocol for all integrations (M + N)

More formally:

Models ⟷ MCP ⟷ Tools

Now:

Integrations = M + N

Instead of:

3 × 5 = 15

we get:

3 + 5 = 8

This is exactly why the notebook keeps emphasizing the move from:

M × N to M + N

03 Lesson 3: Think of MCP Like USB-C

A simple analogy to understand the power of a standard protocol.

MCP is like USB-C for AI tools and models.
One standard connection. Everything just works.

Before USB-C (Fragmented)

Every device needed its own special cable.

Phone	→	Lightning Cable
Camera	→	Mini-USB Cable
Printer	→	USB-B Cable
Headphones	→	3.5mm Audio Cable

✗ Many cables. Not interoperable. Hard to maintain.

After USB-C (Standardized)

One standard cable works for everything.

Phone	→	USB-C Cable
Camera	→	
Printer	→	
Headphones	→	

✓ One standard. Universal. Easy to build and scale.

Similarly, with MCP:

Without MCP

- ✗ GitHub tool for OpenAI
- ✗ GitHub tool for Claude
- ✗ GitHub tool for Gemini

⋮

Many custom integrations.
Hard to build and maintain.

With MCP

- ✓ GitHub MCP Server
- ✓ One integration.
  Any MCP-compatible model can use it.

MCP provides a universal, standard interface between models and tools—
just like USB-C did for devices.

04 Lesson 4: MCP Architecture

MCP has three core components that work together.

Host
The application the user interacts with

↕

Client
The MCP protocol implementation

↕

Server
The tool or data provider

Together, they enable seamless communication between models and tools through MCP.

1. Host

The application the user interacts with.

Responsibilities

- Talks to the LLM / model
- Manages MCP clients
- Manages sessions and context

Think of it as:

Operating System
It provides the environment and coordinates everything.

2. Client

The MCP protocol implementation.

Responsibilities

- Connects to MCP servers
- Discovers capabilities (tools, resources, prompts)
- Invokes tools and fetches resources
- Maintains communication

Think of it as:

Network Driver
It speaks the protocol and handles communication.

3. Server

The tool or data provider.

Responsibilities

- Exposes capabilities
- Provides tools, resources, and prompts
- Executes requests
- Returns results

Think of it as:

Tool Provider
It offers functionality and data to be used.

In short: Host (runs the app) ⟷ Client (speaks MCP) ⟷ Server (provides capabilities)

This simple, modular architecture enables any MCP-compatible host to use any MCP-compatible server.

05 Lesson 5: What Can A Server Expose?

An MCP Server exposes exactly 3 primitives.

An MCP Server exposes exactly 3 primitives:

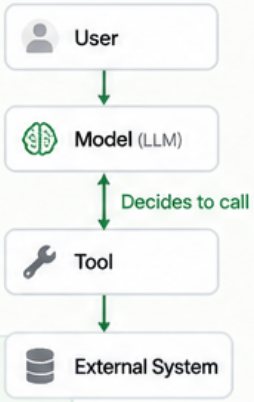
The 3 Primitives	Controlled By
Tools	Model
Resources	Application
Prompts	User

1. TOOLS

Perform actions.
The LLM decides when to call them.

Examples

- create_github_issue()
- send_slack_message()
- run_sql_query()
- create_jira_ticket()



Example Scenario

User: "Create a Jira ticket."
↓
Model decides: create_jira_ticket(...)
↓
Tool executes and returns result.

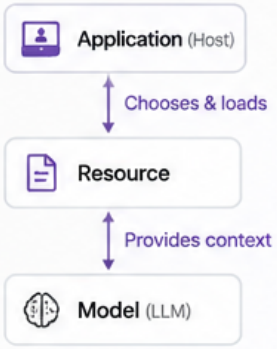
Controlled by: Model

2. RESOURCES

Read-only information.
The application decides what to provide.

Examples

- List documents
- Read configuration
- View git history
- Read database schema
- View API specification



Example Scenario

Host loads database schema
↓
Host injects schema into context
↓
LLM sees schema and uses it.

Controlled by: Application

3. PROMPTS

User-triggered templates.
The user decides when to use them.

Examples

- /explain_code
- /review_pr
- /generate_report
- /summarize_thread

Example Scenario

User clicks: "Review Pull Request"
↓
Server returns a predefined prompt
↓
LLM uses it to generate the response.

Controlled by: User

Easy Rule

Ask: Who decides to fetch or use it?

If the APPLICATION decides → It's a RESOURCE

If the MODEL decides → It's a TOOL

If the USER decides → It's a PROMPT

06 Lesson 6: GitHub Example

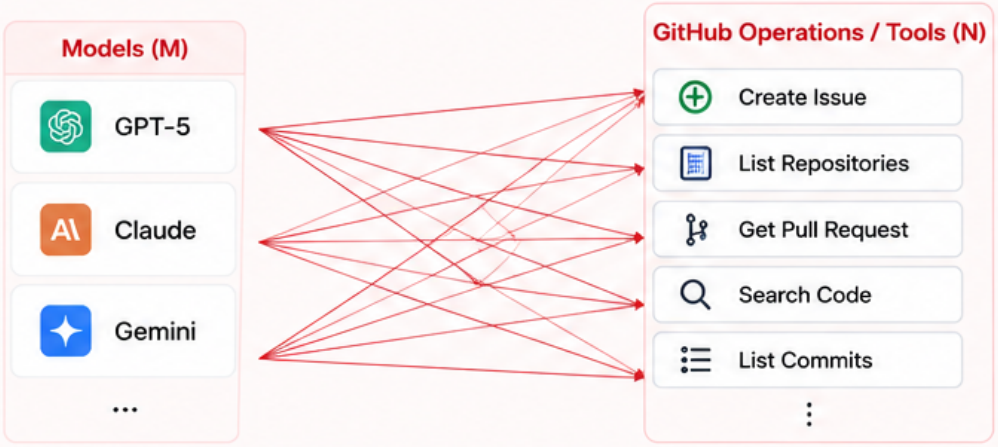
See how MCP eliminates custom work and simplifies integrations.

Goal: Let models (GPT-5, Claude, Gemini, etc.) use GitHub capabilities in a consistent, standard way.

Without MCP

Custom integrations for every model

You build and maintain separate integrations for each model.



- You manually write tool schemas for each operation
- You implement execution, auth, error handling, pagination, rate limiting, etc., for every model
- High maintenance, duplicated work, inconsistent behavior

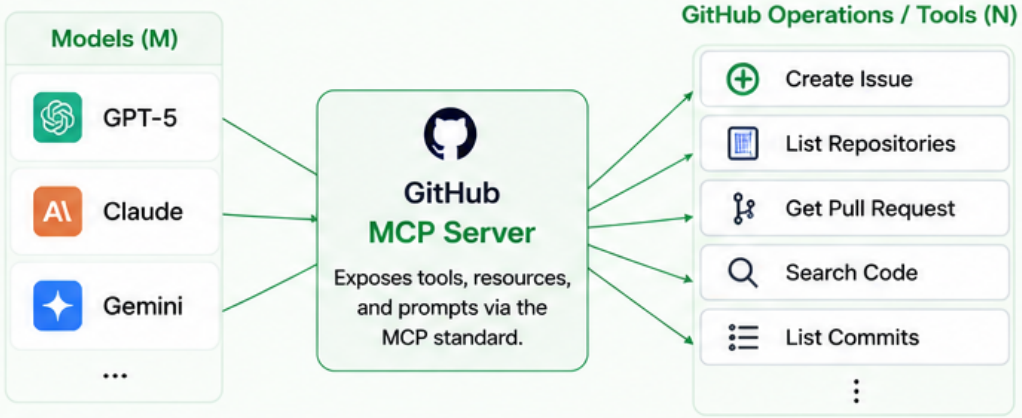
Integrations = M × N

Result:
M × N custom integrations (e.g., 3 models × 5 tools = 15 integrations)

With MCP

One server. All models.

GitHub team builds one MCP Server. Any MCP-compatible model can use it.



- Tools are discovered automatically
- Consistent schemas and behavior across all models
- Only one integration to build and maintain

Integrations = M + N

Result:
M + N integrations (e.g., 3 models + 5 tools = 8 integrations)

The GitHub MCP Server is built once by GitHub, and it works with any MCP-compatible model.

07 Lesson 7: Tool Discovery

How MCP lets models discover tools dynamically instead of hardcoding them.

Tool Discovery is the ability of an MCP Client to ask an MCP Server what tools it has and get the list at runtime.

Traditional Approach (Without MCP)

Tools are hardcoded by developers.

Developer Hardcodes Tools

Developer writes:

```
tool_1
tool_2
tool_3
tool_4
...
```

Model Only Knows What Was Hardcoded

Model can use only these predefined tools.

Any new tool requires code changes and redeployment.

Static. Rigid. Hard to scale.

With MCP (Dynamic Discovery)

Client discovers tools from the Server at runtime.

1. Client Requests Tool List

MCP Client

"What tools do you have?"

2. Server Responds with Tool List

MCP Server

```
[
 { "name": "create_issue",
   "description": "Create
   GitHub issue" },
 { "name": "list_repos",
   "description": "List
   repositories" },
 { "name": "search_code",
   "description": "Search
   code in repos" }
 ...
]
```

3. Client Shares with the Model

Model (LLM)

Model now knows all available tools and can decide which to call.

4. Model Uses Tools via Client

Model → MCP Client → MCP Server

Dynamic. Flexible. Extensible.

Add a new tool on the server → Available to all models automatically.

Why Tool Discovery Matters

No hardcoding

Tools are discovered, not written in code.

Easier extensibility

Add new tools on the server without touching clients or models.

Consistent across models

All MCP-compatible models see the same tool capabilities.

Reduced maintenance

No per-model updates for new or changed tools.

Key Takeaway: MCP enables capability discovery — the foundation for a truly interoperable and scalable ecosystem.

08 Lesson 8: Resources Use URIs

Resources in MCP are identified and accessed using URIs—just like on the web.

A Resource is read-only information provided by the server.

It is identified by a URI. The client (application) decides which resource to fetch.

What is a Resource?

- Read-only information
- Application decides what to load
- Identified by a URI
- Returned by the server

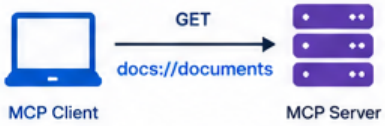
Examples

- docs://documents
  List all documents
- documents://report.pdf
  Get contents of a specific document
- db://schema/public/users
  Get database table schema
- config://app/settings
  Get application configuration

How It Works (Step-by-Step)

1 Client Requests a Resource

The client asks the server for a resource using a URI.



Request (Example)

```
{
  "method": "resources/read",
  "params": {
    "uri":  "docs://documents"
  }
}
```

2 Server Resolves the URI

The server finds the resource that matches the URI.

Server Action

Looks up docs://documents in its data store and prepares the response.

3 Server Returns the Resource

The server returns the resource content.

Response (Example)

```
{
  "contents": [
    {
      "uri": "docs://documents",
      "mimeType": "application/json",
      "text": "[\"report.pdf\",\"notes.txt\",
                \"roadmap.docx\"]"
    }
  ]
}
```

Client Action

Use the resource in the application or provide it to the LLM as context.

Another Example: Fetching a Specific Document

Why URIs?

- Standard and familiar
- Easy to reference
- Hierarchical and logical
- Works across systems and tools

Think of it like REST APIs

- URI = Address
- Resource = What's at that address

Key Takeaway: In MCP, resources are accessed using URIs—just like web resources. The application decides what to fetch, and the server returns read-only information.

One URI. One Resource. Anywhere.

09 Lesson 9: Why MCP Matters for Agents

MCP enables agents, models, and tools to work together through a common standard.

Key Idea

Without MCP, every new model–tool integration is **custom work**.

With MCP, agents can connect to **any MCP-compatible tool** instantly.

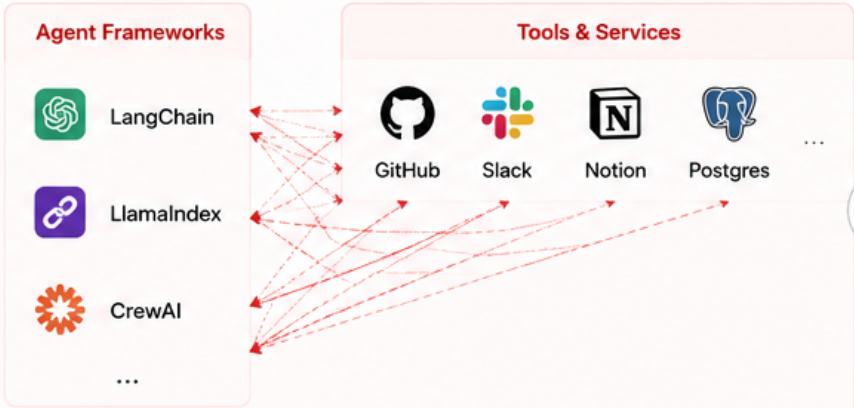
Result

Agents become more capable, extensible, and future-proof.

Build once. Connect anywhere.

Without MCP: Fragmented Ecosystem

Every agent framework and every tool uses its own integration.



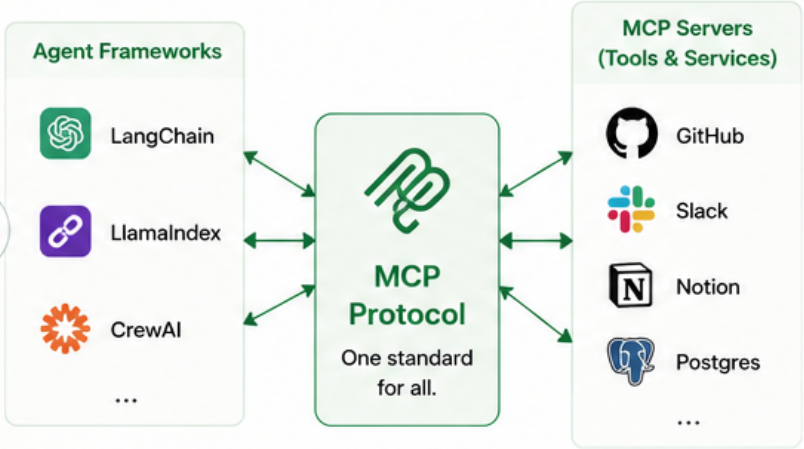
- N × M custom integrations
- High engineering effort to build and maintain
- Inconsistent behavior and capabilities
- Hard to scale and keep up with new tools

Complex. Costly. Hard to Scale.

VS

With MCP: Interoperable Ecosystem

One standard. Any agent can use any MCP-compatible tool.



- M + N integrations (instead of N × M)
- Consistent, predictable, and secure interactions
- Easier onboarding of new tools and models
- Unlocks true composability for agents

Simple. Scalable. Future-Proof.

Why This Matters for Agents

More Capable

Agents can access a broader set of real-world tools and data.

More Flexible

Swap tools, add new ones, and mix capabilities easily.

More Reliable

Standardized interactions improve security, auth, and error handling.

More Future-Proof

New models and tools just work—no custom integration required.

Ecosystem Growth

A shared standard drives innovation for everyone.

Bottom Line: MCP is the connective tissue for the agent ecosystem—turning isolated integrations into a unified, scalable network.
Better for **developers**. Better for **tools**. Best for **agents**.

10 Lesson 10: How This Connects To AI Agents

MCP is the way AI agents discover, understand, and use real-world capabilities.

Big Picture

AI agents need tools, data, and context to act in the real-world.
MCP gives them a standard, reliable way to find and use what they need.

1. The Agent's Need

The agent has a goal and may not know which tools or data can help.

Example Goal

"Find and summarize the latest issues in my GitHub repo."

Agent Capabilities

- Reason and plan
- Ask for help
- Use tools and data
- Learn from results
- Take action

2. How MCP Enables the Agent

1 Discover
Agent asks the MCP server what's available.

2 Understand
Server returns tools, resources, and prompts with context.

3 Decide & Call
Agent decides what to use and calls the right thing.

4 Get Results
Agent receives results and continues the task.

MCP Protocol (Standard Layer)
Standard messages for discovery, invocation, and results

MCP Server (Tools, Resources, Prompts)

Tools (Actions)
- create_issue
- list_issues
- search_code
- ...

Resources (Data)
- repo://issues
- repo://readme
- repo://commits
- ...

Prompts (Templates)
- summarize_issue
- draft_response
- explain_code
- ...

Backends & Systems
GitHub, Databases, APIs, Filesystems, Docs, SaaS apps, and more

3. The Outcome

The agent completes the goal using real-world capabilities.

Example Outcome

"Here's a summary of the latest issues in your repo ..."

Why This Matters

Interoperability
Any agent can use any MCP-compatible server.

Modularity
Servers specialize.
Agents focus on reasoning.

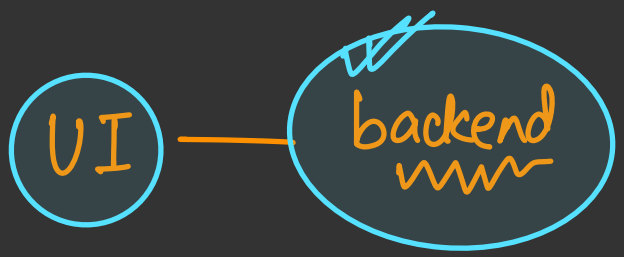
Scalability
Add new capabilities without changing the agent.

Reliability & Security
Standardized interactions with clear boundaries.

Key Takeaway: MCP is the bridge between AI agents and the real world—enabling discovery, understanding, and safe, reliable action at scale.

Discover · Understand · Use

#Doubb

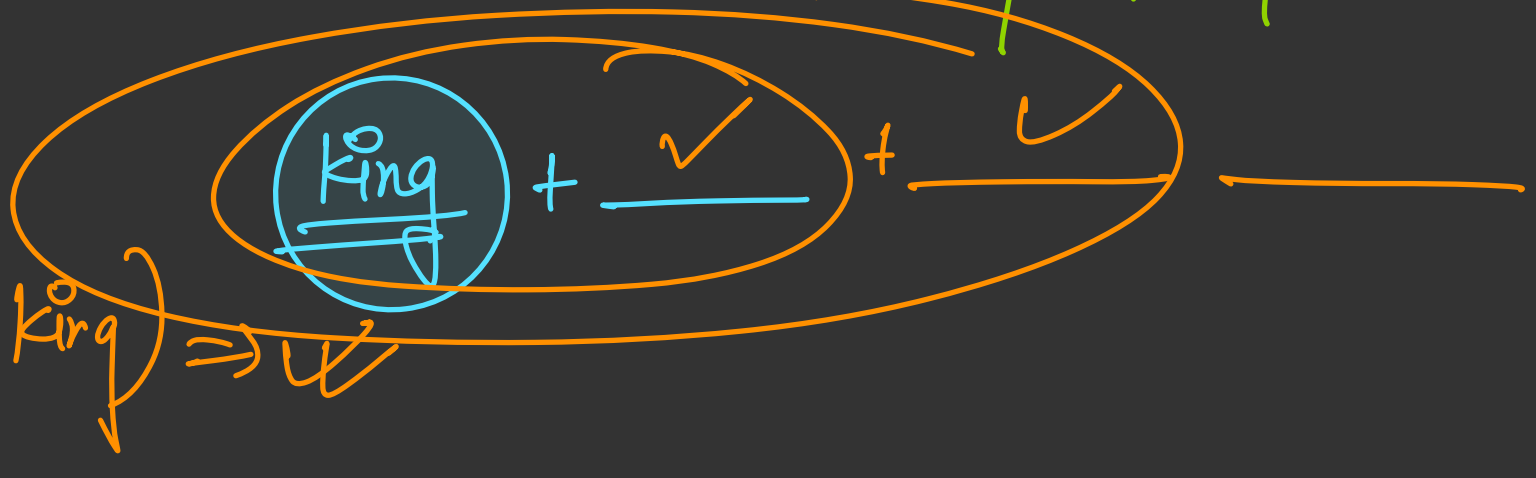


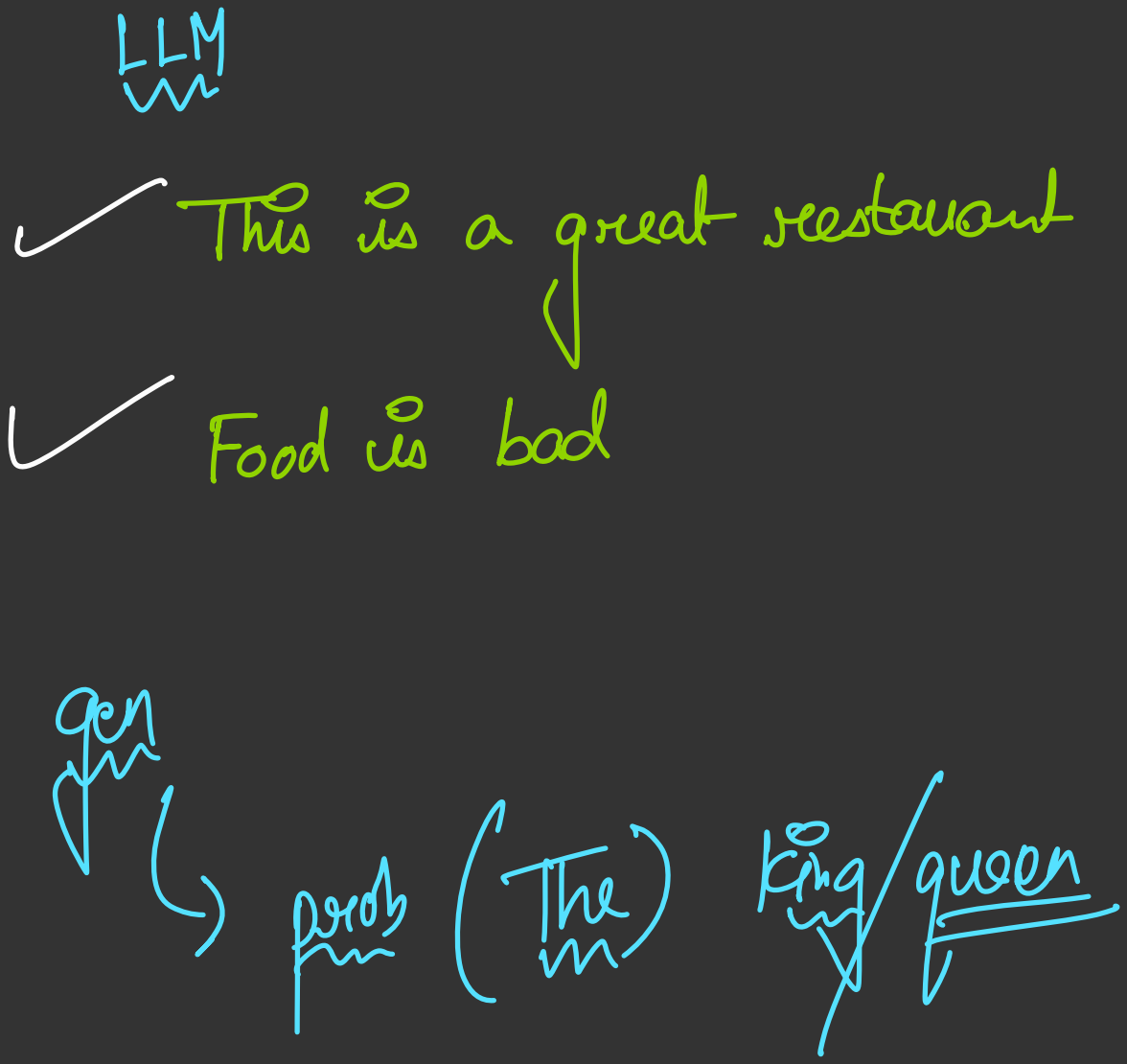
Decoder Only → predict Next token

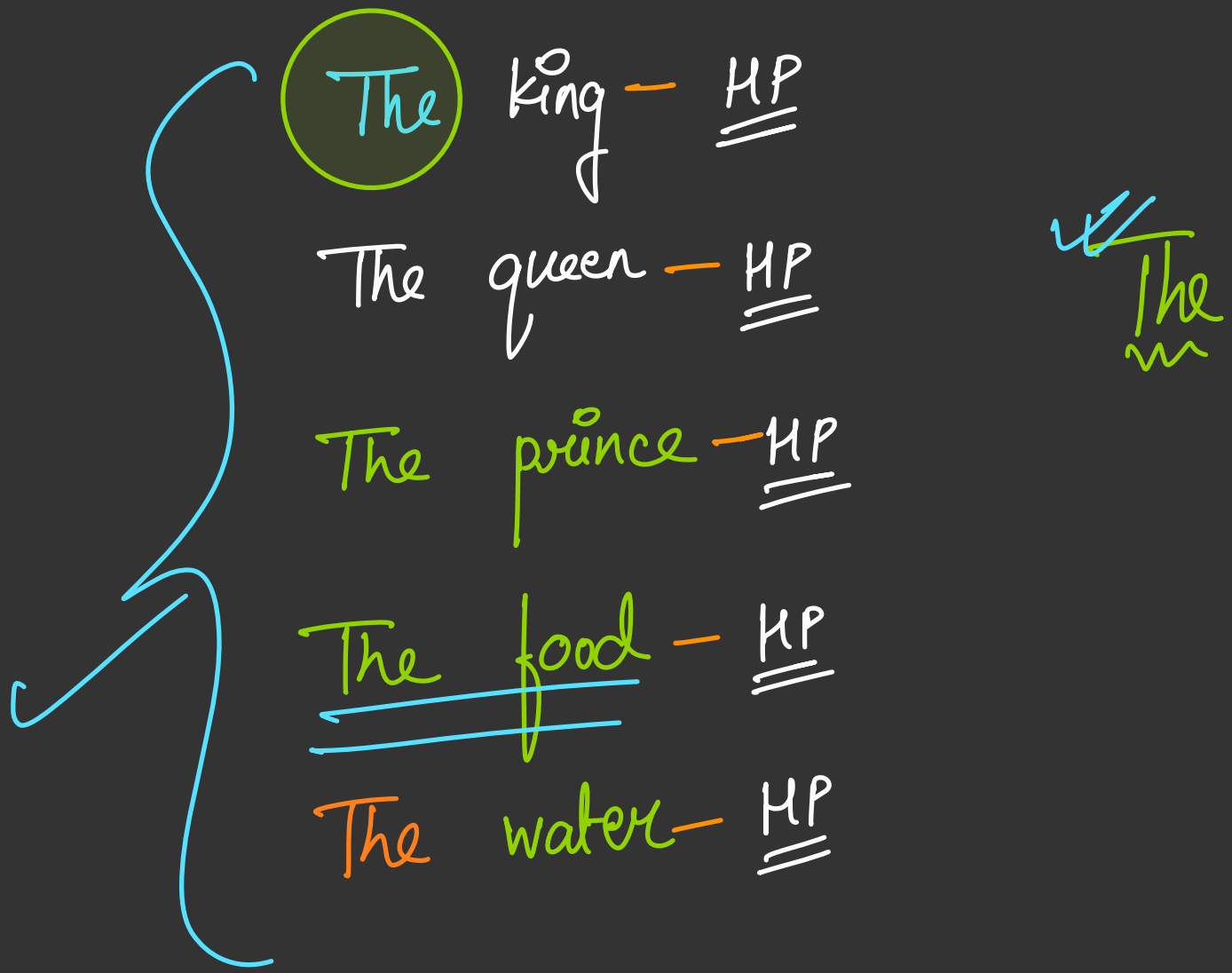
(LLM)

GPT

↳ Beam seach / top-p / top k




LLM
This is a great restaurant
Food is bad
gen
prob (The king/queen


The king – HP
The queen – HP
The prince – HP
The food – HP
The water – HP

The